# Backend Ledger: Learning & Revision Documentation

This document serves as a complete learning guide to understand the **Backend Ledger** application. It is structured to help you revise how a modern Node.js/Express backend works, specifically focusing on how the different pieces of your ledger system connect.

---

## 1. Project Architecture (MVC Pattern)

Your project follows a variation of the **MVC (Model-View-Controller)** pattern combined with routing and middleware.

- **Models ( `src/models` )**: The blueprints for your database. They define the structure of the data you save to MongoDB.
- **Controllers ( `src/controllers` )**: The "brains" of the operations. This is where the actual business logic lives (e.g., deducting money, validating passwords).
- **Routes ( `src/routes` )**: The traffic cops. They map web URLs (like `/api/auth/login` ) to the correct Controller functions.
- **Middleware ( `src/middleware` )**: Security checkpoints. Functions that run *before* the controller to check permissions (like verifying a JWT token).

---

## 2. The Core Components & Their Use Cases

### A. Database Configuration ( `src/config/db.js` )

**Use Case:** Connects your Node environment to the MongoDB database using Mongoose. **Key Concept:** It uses `process.env.MONGO_URI` to keep the database password secure and hidden from source code.

### B. User Authentication Flow

*Files involved:* *`user.model.js`* , *`auth.controller.js`* , *`auth.routes.js`*

1. **Registration ( `POST /api/auth/register` )**:
   - Creates a new user in MongoDB.
   - **Learning Point:** Notice the `userSchema.pre("save", ...)` hook in your model. Before a user is saved, it takes their plain-text password and **hashes** it using `bcryptjs` . This is a critical security practice so you never store bare passwords!
2. **Login ( `POST /api/auth/login` )**:
   - Finds the user and compares the hashed password.
   - **Learning Point:** Upon success, it uses `jsonwebtoken` (JWT) to generate a secure `token` . This token acts as a digital ID card for the user. It is sent back via a Cookie and JSON.

### C. Authentication Middleware ( `src/middleware/auth.middleware.js` )

**Use Case:** Protects private endpoints from anonymous users. **How it works:**

- When a user hits a protected route (like getting their account balance), this middleware runs first.
- It looks for the `token` in the Cookies or the Authorization header.
- It `jwt.verify()` the token securely. If it's valid, it fetches the user details and attaches them to the request ( `req.user = user` ), then calls `next()` to pass control to the actual Controller.

- *Fun Fact:* `authSystemUserMiddleware` does exactly this, but adds one extra check to confirm `user.systemUser === true`.

### D. The Account System

*Files involved:* `account.model.js`, `account.controller.js`, `account.routes.js`

**Use Case:** Represents a virtual wallet for a user. **Key Learning Point - Calculating Balances:** If you look at `account.methods.getBalance` in your model, you'll see a MongoDB **Aggregate**. Instead of having a static `balance: 5000` field on the account that could get out-of-sync, you dynamically calculate the balance by adding up all `CREDIT` entries in the ledger and subtracting all `DEBIT` entries. This guarantees 100% financial accuracy at all times!

### E. The Transaction & Ledger Engine (The Magic)

*Files involved:* `transaction.model.js`, `ledger.model.js`, `transaction.controller.js`

This is the most advanced part of your app. When a user sends money, it follows a strict 10-step process natively heavily relying on **MongoDB Transactions (Sessions)**.

**Why Sessions ( `mongoose.startSession()` )?** If a user transfers ₹1,000 to a friend, you have to do two things:

1. Debit the sender ₹1,000
2. Credit the receiver ₹1,000

Imagine the server crashes right after Step 1 but before Step 2. The sender lost money, but the receiver got nothing! By wrapping these operations in a `session.startTransaction()`, MongoDB treats both steps as a *single package*. If Step 2 fails, MongoDB automatically rolls back Step 1 (nobody loses money). This is called **ACID Compliance**.

**The Double-Entry Ledger:** Rather than simply changing balances, every transfer logs two distinct entries in the `ledger.model.js`:

- A `DEBIT` entry against the sender's account id.
- A `CREDIT` entry against the receiver's account id. This provides a permanent, auditable history of money movement.

---

## 3. Postman Testing (A Revision Guide)

To safely test this on Postman, remember the step-by-step logic:

1. **Identity Creation:** You *must* register/login first to receive an Auth Token.
2. **Passing the Checkpoints:** You *must* supply this token (Under Authorization -> Bearer Token) to access any locked endpoints.
3. **Correct Routing:** Express apps mount routers on base paths. When `app.js` says `app.use("/api/accounts", accountRouter)`, every path inside `accountRouter` is appended to `/api/accounts`.
4. **Action Limits:** A regular user can list accounts ( `GET` ), while a System Admin (updated via your one-off script) is required to run the `/system/initial-funds` injection endpoint.

### Important Notes on the Code Status:

During the previous code audit, a severe bug was found in `user.comparePassword` (it was missing an `await` making login insecure) and the logout model import was missing. These should be fixed

immediately for production.